Graph Databases
Module-5

By

Dr Shivakumar C

Content

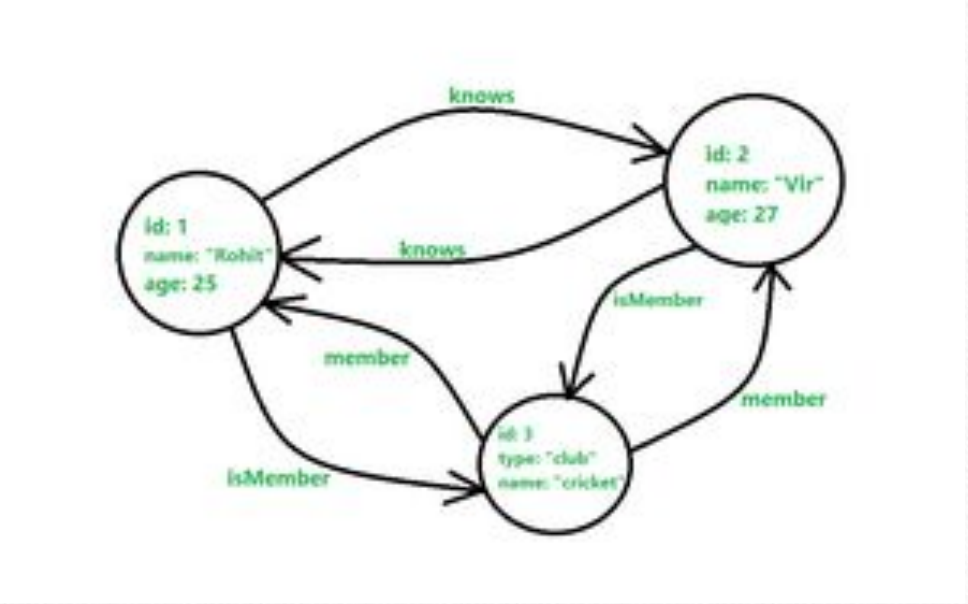
- What Is a Graph Database?
- Neo4j DB Features
- Consistency
- Transactions
- Availability
- Query Features
- Scaling
- Suitable Use Cases
- When Not to Use

What is a Graph Database?

- A graph database (GDB) is a database that uses graph structures for storing data.
- It uses nodes, edges, and properties instead of tables or documents to represent and store data.
- The edges represent relationships between the nodes.
- This helps in retrieving data more easily and, in many cases, with one operation.
- Graph databases are commonly referred to as a NoSQL.

Representation

- The graph database is based on graph theory.
- The data is stored in the nodes of the graph and the relationship between the data are represented by the edges between the nodes.



When do we need Graph Database?

- **1. It solves Many-To-Many relationship problems**
- If we have friends of friends and stuff like that, these are many to many relationships.
- Used when the query in the relational database is very complex.

2. When relationships between data elements are more important

- If there is data element such as user data element inside a graph database
- There could be multiple user data elements but the relationship is what is going to be the factor for all these data elements which are stored inside the graph database.

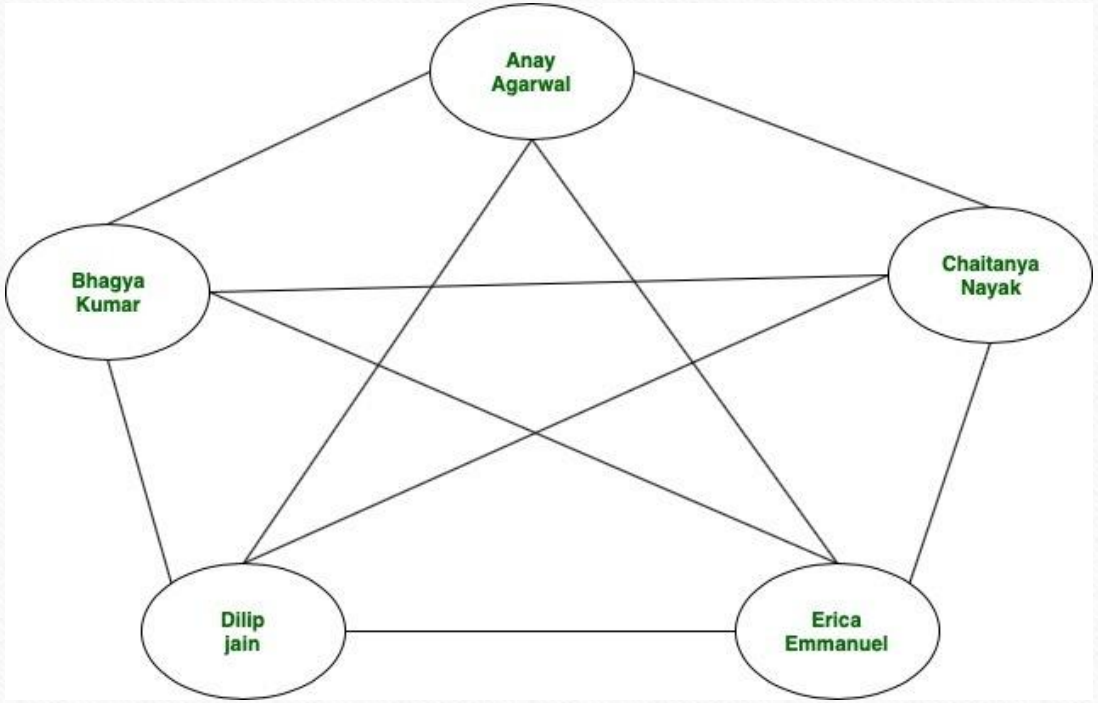
Low latency with large scale data

- When you add lots of relationships in the relational database, the data sets are going to be huge and when you query it,
- The complexity is going to be more complex and it is going to be more than a usual time.
- However, in graph database, it is specifically designed for this particular purpose and one can query relationship with ease.

Why do Graph Databases matter?

- Because graphs are **good at handling relationships**, some databases store data in the form of a graph.
- **Example** We have a social network in which five friends are all connected. These friends are Anay, Bhagya, Chaitanya, Dilip, and Erica.
- A graph database that will store their personal information may look something like this:

Why do Graph Databases matter?



Why do Graph Databases matter?

id	first name	last name	email	phone
1	Anay	Agarwal	anay@example.net	555-111-5555
2	Bhagya	Kumar	bhagya@example.net	555-222-5555
3	Chaitanya	Nayak	chaitanya@example.net	555-333-5555
4	Dilip	Jain	dilip@example.net	555-444-5555
5	Erica	Emmanuel	erica@example.net	555-555-5555

Friend's Table

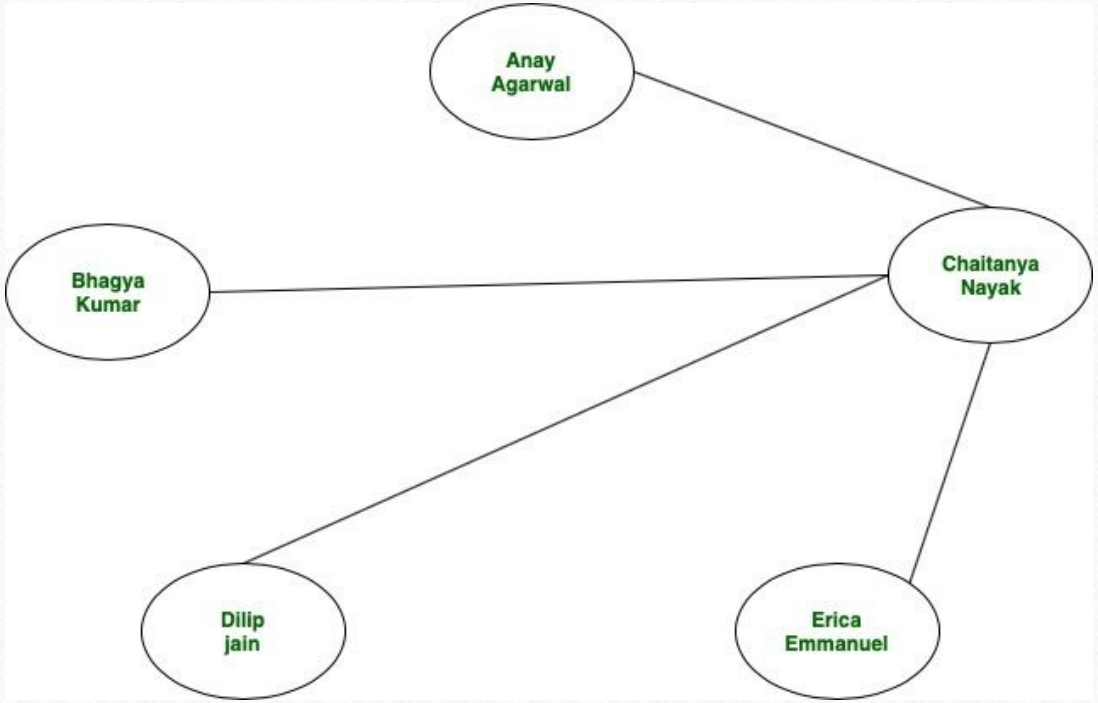
user_id	friend_id
1	2
1	3
1	4
1	5
2	1
2	3
2	4
2	5
3	1
3	2
3	4
3	5
4	1
4	2
4	3
4	5
5	1
5	2
5	3
5	4

To identify Chaitanya's id in the User's table:

id	first name	last name	email	phone
3	Chaitanya	Nayak	chaitanya @example. net	555-333-55 55

user_id	friend_id
3	1
3	2
3	4
3	5

To identify Chaitanya's id in the User's table:



Advantages & Disadvantages

- **Advantages:**

Frequent **schema changes, managing volume of data, real-time query response time**, and **more intelligent data activation requirements** are done by graph model.

- **Disadvantages:**

Note that graph databases aren't always the best solution for an application. We need to assess the needs of application before deciding the architecture.

Limitations

- Graph Databases may not be offering better choice over the NoSQL variations.
- If application needs to **scale horizontally** this may introduces **poor performance.**
- **Not very efficient** when it needs to **update all nodes** with a given parameter.

Popular Graph Databases

- Neo4j is a popular Graph Database.
- Other Graph Databases are Oracle NoSQL Database
- OrientDB
- HypherGraphDB
- GraphBase
- InfiniteGraph
- AllegroGraph.

RDBMS Vs Graph Database

Sr.No	RDBMS	Graph Database
1	Tables	Graphs
2	Rows	Nodes
3	Columns and Data	Properties and its values
4	Constraints	Relationships
5	Joins	Traversal

Features of Neo4j

- **Data model (flexible schema)**
- Neo4j follows a data model named native property graph model.
- Here, the graph contains nodes (entities) and these nodes are connected with each other (depicted by relationships).
- Nodes and relationships store data in key-value pairs known as properties.
- In Neo4j, there is no need to follow a fixed schema. You can add or remove properties as per requirement. It also provides schema constraints.

ACID properties

- Neo4j supports full ACID (
- Atomicity,
- Consistency,
- Isolation,
- Durability) rules.

Scalability and reliability

- You can scale the database by increasing the number of reads/writes, and the volume without effecting the query processing speed and data integrity.
- Neo4j also provides support for replication for data safety and reliability.

Cypher Query Language

- Neo4j provides a powerful declarative query language known as Cypher.
- It uses ASCII-art for depicting graphs.
- Cypher is easy to learn and can be used to create and retrieve relations between data without using the complex queries like Joins.

Built-in web application

- Neo4j provides a built-in Neo4j Browser web application.
- Using this, you can create and query your graph data

Drivers

- Neo4j can work with:
- **REST API** to work with programming languages such as Java, Spring, Scala etc.
- **Java Script** to work with **UI MVC** frameworks such as **Node JS.**
- It supports two kinds of Java **API: Cypher API** and **Native Java API** to develop Java applications.
- In addition to these, you can also work with other databases such as **MongoDB**, **Cassandra,** etc.

Indexing

- Neo4j supports Indexes by using Apache Lucence.

Advantages of Neo4j

- Flexible data model
- Real-time insights
- High availability
- Connected and semi structures data
- Easy retrieval
- Cypher query language
- No joins

Consistency

- Since graph databases are operating on connected nodes, most graph database solutions usually do not support distributing the nodes on different servers.
- There are some solutions, however, that support node distribution across a cluster of servers, such as Infinite Graph.
- Within a single server, data is always consistent, especially in Neo4J which is fully ACID-compliant.
- When running Neo4J in a cluster, a write to the master is eventually synchronized to the slaves, while slaves are always available for read.
- Writes to slaves are allowed and are immediately synchronized to the master

Consistency

- Other slaves will not be synchronized immediately, though—they will have to wait for the data to propagate from the master.
- Graph databases ensure consistency through transactions.
- They do not allow dangling relationships: The start node and end node always have to exist, and nodes can only be deleted if they don't have any relationships attached to them.

Transactions

- Neo4J is ACID-compliant.
- Before changing any nodes or adding any relationships to existing nodes, we have to start a transaction.
- Without wrapping operations in transactions, we will get a NotInTransactionException.
- Read operations can be done without initiating a transaction.

Example

- Transaction transaction = database.beginTx();
- try {
- Node node = database.createNode();
- node.setProperty("name", "NoSQL Distilled");
- node.setProperty("published", "2012");
- transaction.success();
- } finally {
- transaction.finish();
- }

Example of transaction

- In the above code, we started a transaction on the database, then created a node and set properties on it.
- We marked the transaction as success and finally completed it by finish.
- A transaction has to be marked as success, otherwise Neo4J assumes that it was a failure and rolls it back when finish is issued.
- Setting success without issuing finish also does not commit the data to the database.

Availability

- Neo4J, as of version 1.8, achieves high availability by providing for replicated slaves.
- These slaves can also handle writes: When they are written to, they synchronize the write to the current master, and the write is committed first at the master and then at the slave.
- Other slaves will eventually get the update.
- Other graph databases, such as Infinite Graph and FlockDB, provide for distributed storage of the nodes.

Query Features

- Graph databases are supported by query languages such as Gremlin.
- Gremlin is a domain specific language for traversing graphs;
- It can traverse all graph databases that implement the Blueprints property graph.
- Neo4J also has the Cypher query language for querying the graph.
- Outside these query languages, Neo4J allows you to query the graph for properties of the nodes, traverse the graph, or navigate the nodes relationships using language bindings.

Query Features

- Properties of a node can be indexed using the indexing service.
- Similarly, properties of relationships or edges can be indexed, so a node or edge can be found by the value.
- Indexes should be queried to find the starting node to begin a traversal.
- Let's look at searching for the node using node indexing.
- Index<Node> nodeIndex = graphDb.index().forNodes("nodes");

Scaling

- In NoSQL databases, one of the commonly used scaling techniques is sharding, where data is split and distributed across different servers.
- With graph databases, sharding is difficult, as graph databases are not aggregate-oriented but relationship-oriented.
- Since any given node can be related to any other node, storing related nodes on the same server is better for graph traversal.
- Traversing a graph when the nodes are on different machines is not good for performance.
- Knowing this limitation of the graph databases, we can still scale them using some common techniques described by Jim Webber.

Scaling

- Generally speaking, there are three ways to scale graph databases.
- Since machines now can come with lots of RAM, we can add enough RAM to the server so that the working set of nodes and relationships is held entirely in memory.
- This technique is only helpful if the dataset that we are working with will fit in a realistic amount of RAM.

Scaling

- We can improve the read scaling of the database by adding more slaves with read-only access to the data, with all the writes going to the master.
- When the dataset size makes replication impractical, we can shard the data from the application side using domain-specific knowledge.

Suitable Use Cases

Connected Data

- Social networks are where graph databases can be deployed and used very effectively.
- These social graphs don't have to be only of the friend kind; for example, they can represent employees, their knowledge, and where they worked with other employees on different projects.
- Any link-rich domain is well suited for graph databases.

Suitable Use Cases

Routing, Dispatch, and Location-Based Services

- Every location or address that has a delivery is a node, and all the nodes where the delivery has to be made by the delivery person can be modeled as a graph of nodes.
- Relationships between nodes can have the property of distance, thus allowing you to deliver the goods in an efficient manner.
- Distance and location properties can also be used in graphs of places of interest, so that your application can provide recommendations of good restaurants or entertainment options nearby.

Suitable Use Cases

Recommendation Engines

- As nodes and relationships are created in the system, they can be used to make recommendations like "your friends also bought this product" or "when invoicing this item, these other items are usually invoiced." Or, it can be used to make recommendations to travelers mentioning that when other visitors come to Barcelona they usually visit Antonio Gaudi's creations

When Not to Use

- In some situations, graph databases may not appropriate.
- When you want to update all or a subset of entities for example, in an analytics solution where all entities may need to be updated with a changed property graph databases may not be optimal since changing a property on all the nodes is not a straightforward operation.
- Even if the data model works for the problem domain, some databases may be unable to handle lots of data, especially in global graph operations